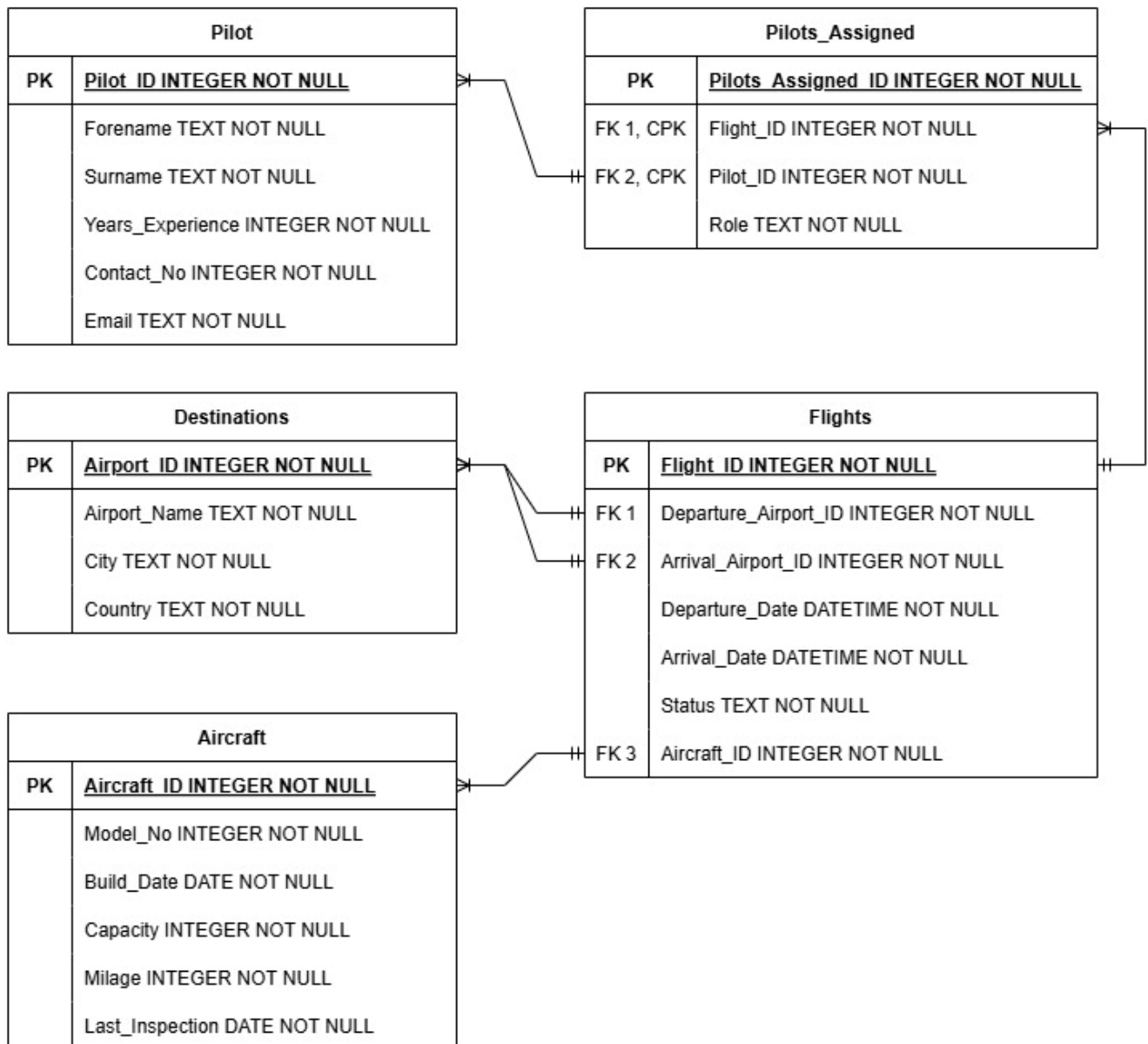


Flight Management Database Documentation

ER Diagram image:



Relational Schema:

Pilot

This table contains all the individual information about each pilot. For a flight management database, you want to know how much experience each pilot has, and how to contact them. This allows staff to determine the most appropriate pilot to fly an aircraft and then notify them that they have been assigned to a new flight.

Each pilot has a Pilot_ID which is the primary key and is linked to the Pilots_Assigned table. This is a many to one link, as many different pilots will be used in the Pilots_Assigned table but each Pilot_ID only refers to one pilot.

Pilots_Assigned

This table enables multiple pilots to be assigned to one flight, and specifies their role (Captain/Co-pilot) on that flight.

The primary key is an ID number (Pilots_Assigned_ID) and the table includes a composite key of Flight_ID and Pilot_ID. This composite key prevents the same pilot from being assigned to the same flight twice. Furthermore this table includes two foreign keys, the first being Pilot_ID as discussed in the Pilot table section, and the second is the Flight_ID. Flight_ID links to the primary key in the table Flights, and many to one connection which enables multiple pilots to be assigned to the one flight.

Destinations

This table contains all the needed information about a destination for a flight management database. It allows for multiple airports within one city, and for multiple different airports across a country. This makes it easy for staff to search for flights to a certain country, or if they had to update a flight to a new destination, they could find alternative airports within that city or country.

This table contains a primary key, Airport_ID, which is linked both the Departure_Airport_ID and the Arrival_Airport_ID within the Flights table. The connection is a many to one, which allows each airport to be assigned as the departure or arrival airport for different flights.

Aircraft

This table contains all the information about each individual aircraft the company flies. This enables us to determine the condition of the aircraft from the date it was built, to how far it has flown and when it was last inspected. We can also use this table to determine how many passengers can fly on that route with the capacity of each plane, therefore allowing for a more optimal selection of aircraft each time a new flight is scheduled.

This table has a primary key of an Aircraft_ID number which is linked with a many to one connection to the Aircraft_ID field in the flights table. This allows each aircraft to be assigned to multiple flights.

Flights

This is the central table where the information leads into. Within this table is each flight that has been created. For each flight you can see where you are going to and coming from, with information from the Destinations table. You can see which aircraft you are flying on, its capacity and condition. You can also see which pilots have been assigned to the flight through the Pilots Assigned table, and through this, determine their years of experience and how to contact them. Finally this table contains the status of each flight, whether it has been scheduled, delayed or cancelled, and when it is expecting to take off and land.

The primary key Flight_ID is linked to the Pilots_Assigned table, and contains 3 foreign keys, Departure_Airport_ID, Arrival_Airport_ID and Aircraft_ID. These links have been discussed above and prevent duplicate information being stored each time a new flight is created.

Functions:

My functions are separated into 4 groups. The first group allows you to interact with the database through flights. This enables the user to search for a specific flight by certain criteria or to update the flight information. The second group allows the user to interact with the database through pilot information. With this they can view a pilot's schedule or assign a pilot to a flight. The third group creates a new flight or deletes an existing one and the fourth group allows the user to see information about the different aircrafts. Within the fourth group the user can see how much each aircraft has flown and when they were last inspected, and update the last inspection date. 3 general functions that were used repeatedly throughout the code are also defined, these functions validate whether a value is in the database, ensure that a date is in the correct format, and list various options that the user can use as inputs when interacting with the database.

~~~ General Functions ~~~

Validate Existence

This function is designed to return true if there is a value that matches the criteria and false if not.

```
query = f"SELECT * FROM {table} WHERE {column} = ? LIMIT 1;"
```

SELECT * returns all values that match the criteria and LIMIT 1 restricts that value to 1 for efficiency. The function takes in cursor, table, column and value as parameters. Table and column are identifiers and added in via curly brackets such that they can be changed dynamically in the query. Whereas value is added in with a placeholder ? as this is a value inputted by the user and as such is vulnerable to potential malicious inputs through SQL injection.

List Options

This function is designed to list all the options that the user can select from. Since many of the inputs from the menu are based on ID numbers, this function displays the ID column and the name attached to it such that the user can easily identify and input their choice.

```
query = f"SELECT {id_column}, {name_column} FROM {table};"
```

All the inputs are identifiers already within the database so there is no risk of malicious input, thus id_column, name_column and table (which are all parameters in the function) are contained within curly brackets. This query finds the id and name columns from a specified table and returns them to the user.

Validate Date Input

This function does not query the database but rather checks whether the date is in the correct format. If it isn't then it prints an error and displays the desired format

~~~ Find a flight ~~~

Find Flight Menu

This prints the different options to search flights or update the flight information and takes the input of 1,2 or 3 to call these functions (3 goes back to the main menu)

Search Flights

This function has 3 different ways to search for a flight, first by a specified time range, second by a destination and third by a specific aircraft.

The first takes the input start_time and end_time and using the validate date input function defined about, ensures they are valid inputs

```
query = """
SELECT Flight_ID, Departure_Date, Arrival_Date, Status FROM
Flights
WHERE Departure_Date BETWEEN ? AND ?;
"""
cursor.execute(query, (start_time, end_time))
```

The query then finds all the flights that have a departure date between the start and end time. It returns the flight ID, departure and arrival date, and the current status which are all contained within the Flights table. Since start and end time are user inputs, ? placeholders have been used as they are dynamic and prevent potential malicious inputs. Since no dynamic identifiers (table/column names in {}) have been used, the query can be contained within the static "..." .

The second function presents the user with another choice, whether they would like to search the destination by airport ID, city or country, taking a number input. Using that input, the function then lists the valid options of input (different cities or airport IDs etc), prompts the user for an input and sets the column variable to the correct column within the database.

```
query_validation = f"SELECT Airport_ID FROM Destinations WHERE
{column} = ? LIMIT 1;"
```

This query then simply finds the correct Airport_ID for that input by searching the appropriate column within the destinations table, and limits the output to 1 to prevent errors.

The user is then prompted to select whether they are looking for arrivals or departures and this sets the direction variable used in the final query.

```
query = f"""
SELECT Flights.Flight_ID, Flights.Departure_Date,
Flights.Arrival_Date, Flights.Status,
Departure.Airport_Name AS From_Airport, Arrival.Airport_Name AS
To_Airport FROM Flights
JOIN Destinations AS Departure ON Flights.Departure_Airport_ID =
Departure.Airport_ID
JOIN Destinations AS Arrival ON Flights.Arrival_Airport_ID =
Arrival.Airport_ID
WHERE Flights.{direction} = ?;
"""
```

This query finds the relevant information about all the flights going either to or from a certain destination. It draws upon 2 different tables, flights and departures and as such all the columns are specified upon which table they refer to. Since Departure and Arrival both reference the Airport ID within the departure table, these columns have been renamed within the query used the AS function. To link the databases and allow the arrival and departure columns within flight to reference the airport ID within departure, the join function has been used. Similarly since destinations is being joined twice, the AS function has been used to specify which column it is being joined too.

The third function within search flights, allows the user to search via a specific aircraft. It first lists the possible aircrafts using the list options function defined above, and then prompts the user for an aircraft, validating that aircraft exists using the validate existence function. This is then used in the following query:

```
query = ""
SELECT Flights.Flight_ID, Flights.Departure_Date,
Flights.Arrival_Date, Flights.Status,
Departure.Airport_Name AS From_Airport, Arrival.Airport_Name AS
To_Airport FROM Flights
JOIN Destinations AS Departure ON Flights.Departure_Airport_ID =
Departure.Airport_ID
JOIN Destinations AS Arrival ON Flights.Arrival_Airport_ID =
Arrival.Airport_ID
WHERE Flights.Aircraft_ID = ?;
""
```

This query is the same as the one above, however the condition for which flight information to use (WHERE function) is determined by the aircraft ID.

Update Flight Information

This function starts with a more detailed list of the different flights, to allow the user to know what flights they can select from.

```
query_flights = ""
SELECT Flights.Flight_ID, Departure.City AS From_City, Arrival.City AS
To_City,
Flights.Departure_Date, Flights.Status FROM Flights
JOIN Destinations AS Departure ON Flights.Departure_Airport_ID =
Departure.Airport_ID
JOIN Destinations AS Arrival ON Flights.Arrival_Airport_ID =
Arrival.Airport_ID;
""
```

This query uses the same AS and JOIN logic as the previous two queries, allowing the different cities the flight is going to and from to be referenced between tables. This was required such that the cities that the flight is going to and from can be displayed as opposed to the airport ID, making the program more user-friendly and accessible. There are no specified conditions on which flights to return, and so this query will display all the different flights in the system.

Using this information, the user then inputs the flight ID that they would like to update. The input is then validated using the validate existence function. The user is then prompted as to whether they would like to update the status of the flight or the destination.

If they choose to update the status, then they are prompted for which status they would like to update. This is checked to ensure it is either, Scheduled, Delayed or Cancelled. That input is then used in the following query

```
query_update_status = "UPDATE Flights SET Status = ? WHERE Flight_ID = ?;"
```

This query changes the status for a certain flight to the new status defined by the user's input.

If they choose to update the destination, then after listing the destination options, the user is prompted to input a destination ID which is checked using the validate existence function.

```
query_update_destination = "UPDATE Flights SET Arrival_Airport_ID = ? WHERE Flight_ID = ?;"
```

Similar to the last query, this changes the arrival airport to the new selected destination for the selected flight.

~~~ Find a pilot ~~~

Find Pilot Menu

Similar to the find flight menu, this prints the different options of function and takes a number input between 1 and 3, which then calls the respective function.

View Pilot Schedule

This function starts by displaying all the different pilots from which you can input.

```
query_pilots = ""  
SELECT Pilot_ID, Name, Years_Experience  
FROM Pilot ORDER BY Years_Experience DESC;  
""
```

It selects the pilots ID, name and years of experience from the pilots table, then using the ORDER BY command, orders that list based on the years of experience.

The user is then prompted as to whether they would like to search via the ID or pilot name. If they search via ID, that ID is checked with the validate existence function. If they search by name, the function runs the following query.

```
query_pilot = "SELECT Pilot_ID FROM Pilot WHERE Name = ?;"
```

This simply returns the pilot ID which gets assigned to a variable used in the next query, and prints an error if the pilot is not found.

The pilot's schedule is then queried.

```

        query_schedule = """
        SELECT Flights.Flight_ID, Departure.City AS From_Airport, Arrival.City
AS To_Airport,
        Flights.Departure_Date, Flights.Arrival_Date, Flights.Status FROM
Flights
        JOIN Pilots_Assigned ON Flights.Flight_ID = Pilots_Assigned.Flight_ID
        JOIN Destinations AS Departure ON Flights.Departure_Airport_ID =
Departure.Airport_ID
        JOIN Destinations AS Arrival ON Flights.Arrival_Airport_ID =
Arrival.Airport_ID
        WHERE Pilots_Assigned.Pilot_ID = ?;
        """

        cursor.execute(query_schedule, (pilot_id,))

```

This query uses similar logic to before, however now it is joining the pilots assigned table which contains the pilots assigned to each flight, with the flights table to find the timings of their flights, the status and the departure and arrival destination ID which are then linked to the city by joining it to the destinations table. This final join is to make the result more readable to the user as the result is displayed with the city rather than an arbitrary ID value.

Assign Pilot to Flight

First this function checks if the user would like to assign a pilot to an existing flight, or if they would like to create a new flight. If they choose to create a new flight, it will link to the create new flight function discussed below. If they choose to assign to an existing flight, it will first run this query.

```

        query_flights = """
        SELECT Flights.Flight_ID, Departure.City AS From_Airport,
Arrival.City AS To_Airport, Flights.Departure_Date, Flights.Status,
        (SELECT COUNT(*) FROM Pilots_Assigned WHERE
Pilots_Assigned.Flight_ID = Flights.Flight_ID) AS Pilot_Count FROM Flights
        JOIN Destinations AS Departure ON Flights.Departure_Airport_ID =
Departure.Airport_ID
        JOIN Destinations AS Arrival ON Flights.Arrival_Airport_ID =
Arrival.Airport_ID;
        """

```

This function joins the departure and arrival airport ID from flights with destinations to return the city name as before, it also counts the number of pilots that have been assigned to the same flight ID by using the COUNT all (*) function in pilots assigned. The user can then see before deciding which flight id to choose, the flight id, where it is going to and from, when it leaves, the status of the flight and how many pilots are already assigned to that flight. The flight ID that is then chosen is validated as before using the validate existence function.

The function then, using the list options function, displays all the available pilots and their ID such that the user can select a pilot to assign to a flight. Their input is also then validated using the validate existence function.

The function then runs a check to ensure the pilot is not already assigned to that flight using the following query.

```
query_check = """
SELECT 1 FROM Pilots_Assigned WHERE Flight_ID = ? AND Pilot_ID = ?;
"""
cursor.execute(query_check, (flight_id, pilot_id))
```

If this function returns a value, then it means that pilot is already assigned to this flight.

If the function does not return a value, then the user is prompted to enter a role for the pilot. This is also checked to ensure that it is either captain or co-pilot. Provided the user enters a valid role, the function then performs the following query.

```
query_assign = """
INSERT INTO Pilots_Assigned (Flight_ID, Pilot_ID, Role)
VALUES (?, ?, ?);
"""
cursor.execute(query_assign, (flight_id, pilot_id, role))
```

The INSERT INTO function adds a new entry into pilots assigned using the flight id, pilot id and role defined about. Again the placeholder ? is used to prevent SQL injection as an added precaution since flight id, pilot id and role, however these values have already all been validated and checked before being entered into the query.

~~~ Create or Delete a flight ~~~

Add or Remove a Flight Menu

This prints the options of creating a new flight, deleting a flight ore returning to the main menu. The user inputs either 1,2 or 3 to call these functions.

Create a new flight

To add a new flight, the user is required to enter valid inputs for all the fields within the flights table and within the pilots assigned table.

The first step is to choose departure and arrival airports. Using the list options function, the different airport ID and names are displayed for the user to choose from. They are then prompted to enter an input for both the departure and arrival airports, these inputs are validated using the validate existence function. An additional check is included that looks to see if the arrival airport is the same as the departure airport.

The next step is for the user to specify the time of the flight. They are prompted to enter the date and time of the flight departing and arriving, which using the validate date input function is then checked. A further check is then performed to ensure that the arrival date is after the departure date.

The user then is prompted to select an aircraft, these are listed in the terminal for the user to choose from using the list options function. This is again then validated using the validate existence function.

The user is then asked to assign a captain and co-pilot. The function first queries the pilots database to show the available pilot.


```
query_pilots = """
SELECT Pilot_ID, Name, Years_Experience FROM Pilot
ORDER BY Years_Experience DESC;
"""
```

This query simply displays the pilot id, name and years of experience from the pilot table and orders the entries by the years of experience.

Similar to the logic for the departure and arrival selection, the user is prompted to enter a pilot ID for the role of captain and for co-pilot, which are checked to ensure they are in the database and they are not the same person.

Using all these variables, the function now performs the following query to add a new flight.

```
query_flight = """
INSERT INTO Flights (Departure_Airport_ID, Arrival_Airport_ID,
Departure_Date, Arrival_Date, Status, Aircraft_ID)
VALUES (?, ?, ?, ?, 'Scheduled', ?);
"""

cursor.execute(query_flight, (departure_airport, arrival_airport,
departure_date, arrival_date, aircraft_id))
```

Again whilst all inputs have been validated, placeholders have been used as good practice to avoid malicious inputs. This query uses the INSERT INTO function to enter all the variables we defined earlier a new complete entry into the flights table.

Using cursor.lastrowid, we find the ID of this newly created flight, which is used to complete the new entries into the pilots assigned database.

```
query_assign_pilot = """
INSERT INTO Pilots_Assigned (Flight_ID, Pilot_ID, Role)
VALUES (?, ?, ?);
"""

cursor.execute(query_assign_pilot, (flight_id, captain_id,
'Captain'))
cursor.execute(query_assign_pilot, (flight_id, co_pilot_id, 'Co-
Pilot'))
```

Similar to the previous query, this adds a complete entry into the pilots assigned database, and this query is run twice for both the captain and the co-pilot.

Delete a flight

This function allows the user to select a flight from the database and then delete it, which removes it from the flights table and pilots assigned table. It starts by displaying all the flights that the user can delete.

```
query_flights = """
SELECT Flights.Flight_ID, Departure.City AS From_City,
Arrival.City AS To_City,
Flights.Departure_Date, Flights.Arrival_Date, Flights.Status FROM
Flights
JOIN Destinations AS Departure ON Flights.Departure_Airport_ID =
Departure.Airport_ID
```

```
JOIN Destinations AS Arrival ON Flights.Arrival_Airport_ID =
Arrival.Airport_ID;
"""
```

As before this selects the flight ID, departure and arrival city, departure and arrival date and time and the flight status using the join function to connect the departure and arrival airport to the relevant city.

Choosing a flight ID from this list, the input is validated using the validate existence function. The user is then prompted for confirmation as to whether they would like to delete the flight. If they enter yes, the following queries are run.

```
# Step 4: Delete from Pilots_Assigned table
query_delete_pilots = "DELETE FROM Pilots_Assigned WHERE Flight_ID = ?;"
cursor.execute(query_delete_pilots, (flight_id,))

# Step 5: Delete from Flights table
query_delete_flight = "DELETE FROM Flights WHERE Flight_ID = ?;"
cursor.execute(query_delete_flight, (flight_id,))
```

These queries use the DELETE function to remove all instances from pilots assigned and flights where the flight ID matches the ID entered by the user.

~~~ View Aircraft Stats ~~~

View Aircraft Menu

Similar to the other menu functions listed above, this presents the user with the different functions to call that relate to the aircraft.

View Maintenance Schedule

This function is designed to show the user when each aircraft was last inspected and whether it is due an inspection.

```
query = """
SELECT Aircraft.Aircraft_ID, Aircraft.Model_No,
Aircraft.Last_Inspection,
DATE(Last_Inspection, '+365 days') AS Next_Maintenance FROM
Aircraft;
"""
```

This query has the additional functionality of creating a new date value to store when the aircraft should next be inspected. This next maintenance value is 365 days after the last inspection date. This value is then compared to the current date and if the date it should next have an inspection is before the earlier than the current date, it displays "OVERDUE" in the output.

Update Last Inspection

This function allows the user to input a new inspection that has been performed on an aircraft by updating the last date it was inspected. The user first is displayed the different aircrafts using the list options function, and after inputting an aircraft ID, the value is then validated. They are then prompted as to whether they would like to update the date to today's date or an alternate

date. The function then either takes `datetime.now()` or a new date inputted by the user which has been validated using `validate date input`.

```
query = "UPDATE Aircraft SET Last_Inspection = ? WHERE Aircraft_ID = ?;"
cursor.execute(query, (new_date, aircraft_id))
```

Using these values, the function queries the database, using the UPDATE function to update the last inspection date to the new date for the given aircraft

View Aircraft Usage

This function takes no new inputs but runs the following query to count the number of flights each aircraft has taken.

```
query = """
SELECT Aircraft.Aircraft_ID, Aircraft.Model_No,
COUNT(Flights.Flight_ID) AS FlightCount FROM Aircraft
LEFT JOIN Flights ON Aircraft.Aircraft_ID = Flights.Aircraft_ID
GROUP BY Aircraft.Aircraft_ID;
"""
```

This query uses a left join so that when joining the two tables, any aircrafts that do not have flights will not be dropped and links the aircraft table with the flights table. This allows the query to use a count function associated with the flights table, that used in conjunction with the group by function, tells the query to count the number of flight ID values there are for each aircraft ID.

Interacting with the database:

Example to find a specific flight

```
cjogi@DellXPS MINGW64 ~/OneDrive/Documents/Uni Year 5/1. Databases/Coursework
$ python Flight_Management.py

=== Flight Management System ===
1. Find a Flight
2. Find a Pilot
3. Create or Delete a Flight
4. View Aircraft Stats
5. Exit
Enter your choice: 1

=== Find a Flight ===
1. Search Flights
2. Update Flight Information
3. Back to Main Menu
Enter your choice: 1

Search Flights by:
1. Time Range
2. Destination
3. Aircraft
Enter your choice: 2
```

```
Search Destination by:
1. Airport ID
2. City Name
3. Country Name
Enter your choice: 2
Airport_ID: 1, City: London
Airport_ID: 2, City: New York
Airport_ID: 3, City: Paris
Airport_ID: 4, City: Tokyo
Airport_ID: 5, City: Dubai
Airport_ID: 6, City: Frankfurt
Airport_ID: 7, City: Singapore
Airport_ID: 8, City: Sydney
Airport_ID: 9, City: Cape Town
Enter city name: Tokyo
```

```
Search Flights for:
1. Arrivals
2. Departures
Enter your choice: 1

Flights:
Flight ID: 3, From: Charles de Gaulle Airport, To: Tokyo International Airport,
Departure: 2024-12-03 10:00, Arrival: 2024-12-03 18:00, Status: Scheduled

=== Find a Flight ===
1. Search Flights
2. Update Flight Information
3. Back to Main Menu
Enter your choice: 3

=== Flight Management System ===
1. Find a Flight
2. Find a Pilot
3. Create or Delete a Flight
4. View Aircraft Stats
5. Exit
Enter your choice: 5
```

Reflection:

To determine the structure of the database, I started out by listing any potential fields. With this list I then was able to go through and spot fields that could be potentially repeated. This led to a natural clustering of various fields, such as all of those that relate to a destination that if they were to fly to/from that destination again we would have to reinput. With the tables now roughly defined I was able to work out how they would be efficiently connected together. For my database, I have the Flights table as the central table which stores each different flight and draws on information about the various destinations, pilots and aircrafts from the surrounding tables. One issue I had at this stage was how to deal with multiple different pilots being assigned to the same flight (captain/co-pilot). This led to the creation of the pilots assigned

table which enables Flight_ID to be joined to multiple pilots with their roles defined in the pilots assigned table. This created a new potential issue where the same pilot could be assigned to the same flight, to solve this a composite key was included.

The first issue I encountered in creating my functions was how to join the flights table with the destinations table as arrival airport ID and departure airport ID both linked to the airport ID primary key in destinations. Using the AS function I was able to rename the columns within the query which enabled me to join the two databases. In my first set of test, I found it very easy to input an invalid entry and cause the system to throw an error. This led to writing tests for each input to ensure that it was valid, either by checking it was in the correct format (particularly for date time inputs) or by checking it was able to find that value in the database. With many of these checks being repeated, I refactored my code to create general check functions that I could call within each function. Furthermore, with the addition of a delete function, it was now possible for certain tables to not contain any information, and so additional checks were added into the system that ensured the databases were not empty.

The next issue I found within my tests was I wasn't always sure what was classified as a valid input, as I didn't know ID values were in the database. Before any input that requires the user to reference a particular entry in the database, the program will list all the possible valid inputs. For the more basic requests where it is just querying a certain column in a particular table I was able to create a general list function that kept the code clean. However for many of the functions that required the user to select a flight, displaying an airport ID was still not intuitive, which led to the creation of more complicated queries that linked these ID's to the relevant cities from the destinations table. One consideration at this stage was that displaying all the available data such as the names of pilots and their years of experience could create some privacy and security issues. A further development of this system could involve different levels of clearance or passwords that are required to view certain information. This database only has 10-15 entries per table and so listing all valid inputs for a given criteria is still feasible without creating unmanageable long lists of entries. As more data is entered into the system, further specification of search results could be required before valid inputs are printed. For example they might only be able to see pilots who are already in that city, or have to specify a 6 month window to view flights by destination.

The final consideration was the risk of malicious inputs or SQL injection. Where the user was able to enter in a value that would form part of the SQL query, placeholder "?" were used. Furthermore, each input has been checked to ensure that it is a value that the machine is expecting. If the query is requiring a variable that is assigned in the function, then simple {} are used to enhance the readability of the code. An example of a variable assigned in the function is if the user is selecting whether to search by ID or city, which is checked using an if statement, that if it returns true sets the variable to the column name. The last set of refactoring involved splitting the functions into four groups and creating menus for each to improve clarity for the user in the main menu.

Overall I found this to be a valuable learning experience, particularly in how to design an intuitive and efficient database, thinking of different ways that the user might want to interact with the program or potentially break it and how to protect against SQL injection. There were many further developments I would have been interested in adding, such as ensuring all pilots and aircraft were in the correct destinations for each flight, however due to time constraints I was not able to implement.